

STEM OPT Training Program

Weekly Training Report

Week 3

Trainee Name	Arun Gampala
Technology	Java
Project	Task Management System
Role	Frontend Developer
Week	Week 3 — June 1 to June 5, 2026
Company	Hashclick Solutions
HR Email	hr@hashclicksolutions.com
GitHub Repository	https://github.com/arungampala369-dotcom/hashclick-week3-java

1. Overview

This report covers the Week 3 training tasks assigned by Hashclick Solutions as part of the STEM OPT Training Program. This week involved a real-time project simulation — building a full-stack Task Management System using Spring Boot for the backend and HTML/CSS/JavaScript for the frontend.

The project demonstrates practical application of concepts learned in previous weeks (Java, Spring Boot, CRUD) and adds new layers including JWT authentication, Spring Security, and frontend-backend integration — simulating real-world development experience.

2. Assigned Tasks

2.1 Primary Tasks

- Develop login and authentication system with JWT
- Implement task assignment and tracking features
- Integrate backend REST APIs with frontend
- Build responsive UI with HTML, CSS, and JavaScript
- Attend daily standup meetings and follow Agile methodology
- Push code to GitHub with daily commits

2.2 Technologies Used

Technology	Details
Language	Java 17
Backend Framework	Spring Boot 3.2.5
Security	Spring Security + JWT (jjwt 0.11.5)
Database	H2 In-Memory Database
ORM	Spring Data JPA
Frontend	HTML5 + CSS3 + Vanilla JavaScript
Build Tool	Apache Maven 3.9.16
Testing	Postman + Browser

3. Project Structure

```

WeekThreeJava/ ├── pom.xml └── src/main/ ├── java/com/hashclick/taskmanager/ │ │ ├── TaskManagerApplication.java │ │ ├── controller/ (AuthController, │ │ │ TaskController) │ │ ├── service/ (AuthService, TaskService, │ │ │ UserDetailsServiceImpl) │ │ ├── repository/ (UserRepository, TaskRepository) │ │ ├── model/ (User, Task) │ │ ├── security/ (SecurityConfig, JwtUtil, │ │ │ JwtFilter) │ │ ├── dto/ (AuthRequest, AuthResponse, RegisterRequest) │ │ └── resources/static/ │ │ ├── login.html │ │ ├── register.html │ │ └── dashboard.html │ │ ├── css/style.css │ │ └── js/ (auth.js, dashboard.js)

```

3.1 Layer Responsibilities

Layer	Class	Responsibility
Security	SecurityConfig	JWT filter chain, route protection
Security	JwtUtil / JwtFilter	Token generation and validation
Controller	AuthController	Register and Login endpoints
Controller	TaskController	CRUD endpoints for tasks
Service	AuthService	User registration and authentication
Service	TaskService	Task business logic
Frontend	auth.js	Login/Register forms with API calls
Frontend	dashboard.js	Task CRUD, filter, stats update

4. API Endpoints

Method	URL	Description	Auth
POST	/api/auth/register	Register new user	No
POST	/api/auth/login	Login and get JWT token	No
GET	/api/tasks	Get all tasks	JWT
POST	/api/tasks	Create new task	JWT
PUT	/api/tasks/{id}	Update task	JWT
DELETE	/api/tasks/{id}	Delete task	JWT

5. Daily Progress

Day	Date	Work Done
Monday	June 1, 2026	Project setup, Spring Boot scaffold, JWT dependencies, Security config
Tuesday	June 2, 2026	User model, Register and Login APIs, JWT token generation and validation
Wednesday	June 3, 2026	Task model, CRUD APIs, frontend HTML/CSS pages — Login, Register, Dashboard
Thursday	June 4, 2026	Frontend JS — API integration, task add/edit/delete/filter, stats dashboard
Friday	June 5, 2026	Testing, bug fixes, GitHub push, documentation and weekly report

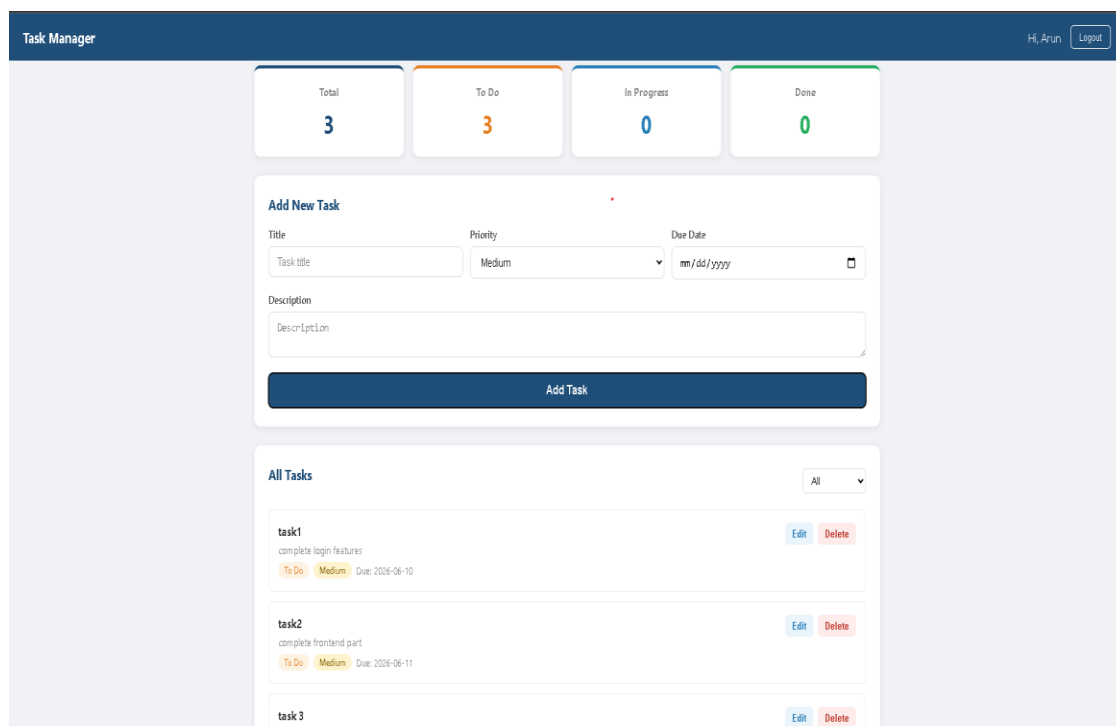
6. Key Learnings

- JWT authentication provides stateless, secure API protection without server-side sessions
- Spring Security filter chain intercepts requests before they reach controllers
- Frontend-backend integration requires matching API contracts (JSON structure, HTTP methods)
- HTML/CSS/JavaScript can build fully functional UIs without any framework
- localStorage is used to persist JWT tokens across page reloads
- Serving frontend from Spring Boot static folder eliminates CORS issues entirely
- Agile methodology — daily standup, incremental delivery, and consistent commits
- Error handling must be implemented on both frontend (user-facing) and backend (API responses)

7. Application Screenshots

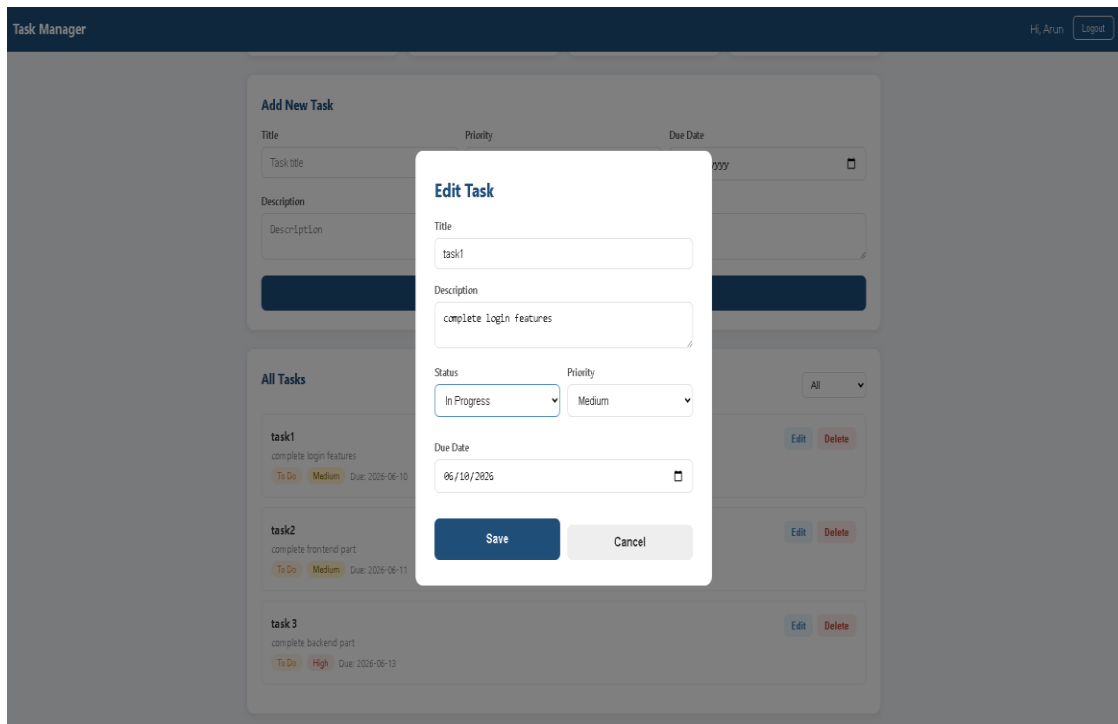
The following screenshots demonstrate the working Task Management System tested in the browser at <http://localhost:8080>.

7.1 Dashboard — Task List with Stats



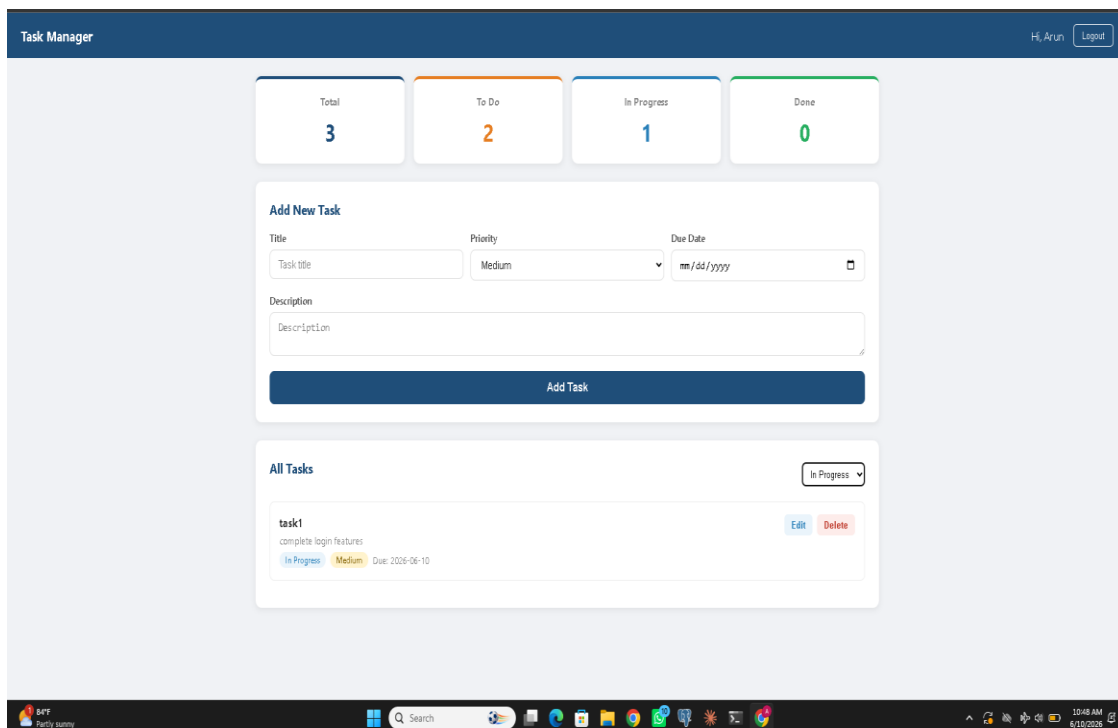
Dashboard showing 3 tasks with stats cards for Total, To Do, In Progress and Done. Tasks display title, description, priority badge and due date with Edit and Delete buttons.

7.2 Edit Task Modal



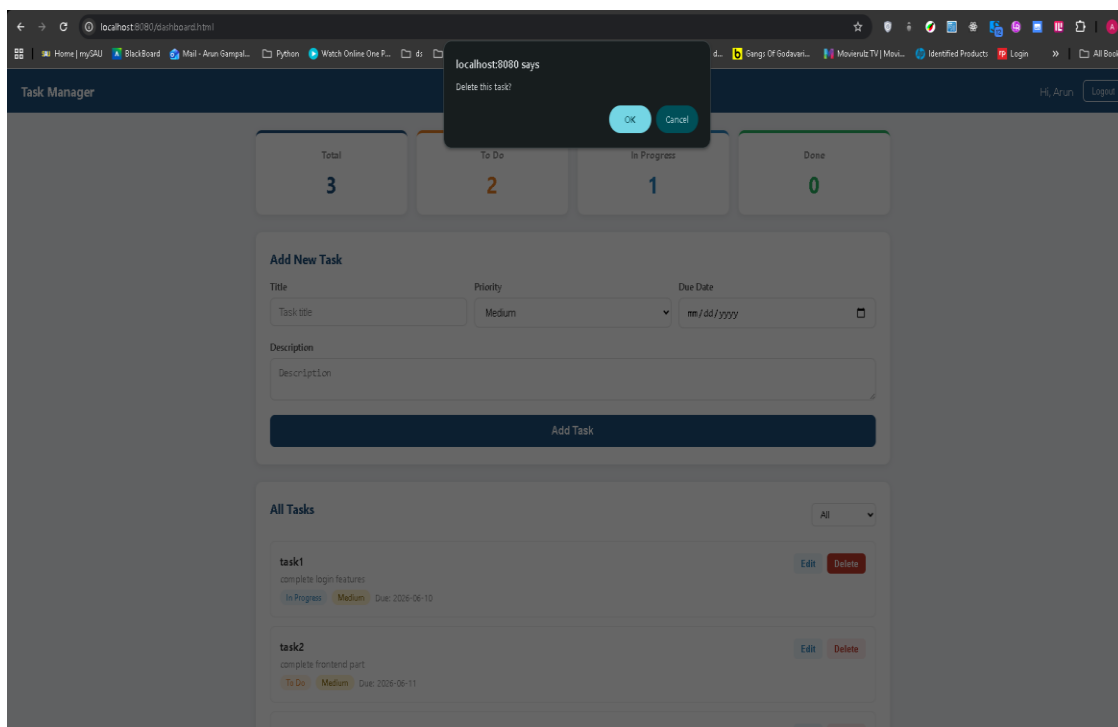
Edit Task modal opens when the Edit button is clicked. The user can update title, description, status (To Do / In Progress / Done), priority and due date before saving.

7.3 Filter by Status — In Progress



Tasks filtered by In Progress status showing only task1. The stats cards update to reflect the current filter, confirming the status was successfully updated via the edit modal.

7.4 Delete Confirmation Dialog



Delete confirmation dialog appears when the Delete button is clicked, asking the user to confirm before permanently removing the task from the database.

8. GitHub Repository

- Repository: <https://github.com/arungampala369-dotcom/hashclick-week3-java>
- All source code committed with meaningful daily commit messages
- README file documenting project setup, structure and API endpoints
- Frontend and backend code maintained in single repository

9. Declaration

I hereby declare that all the work mentioned in this report was completed by me independently as part of the STEM OPT training program at Hashclick Solutions. The code, documentation, and testing were done with consistent daily progress throughout the week.

Signature: Arun Gampala

Name: Arun Gampala

Email: arun@hashclicksolutions.com

Program: STEM OPT Trainee — Java

Company: Hashclick Solutions

Submission Date: June 5, 2026